

Activity 16

Ice roads —*Steiner trees*

Summary

Sometimes a small, seemingly insignificant, variation in the specification of a problem makes a huge difference in how hard it is to solve. This activity, like the Muddy City problem (Activity 9), is about finding short paths through networks. The difference is that here we are allowed to introduce new points into the network if that reduces the path length. The result is a far more difficult problem that is not related to the Muddy City, but is algorithmically equivalent to the cartographer's puzzle (Activity 13) and Tourist Town (Activity 14).

Curriculum Links

- ✓ Mathematics – Position and orientation
- ✓ Mathematics – Logical reasoning

Skills

- ✓ Spatial visualization
- ✓ Geometric reasoning
- ✓ Algorithmic procedures and complexity

Ages

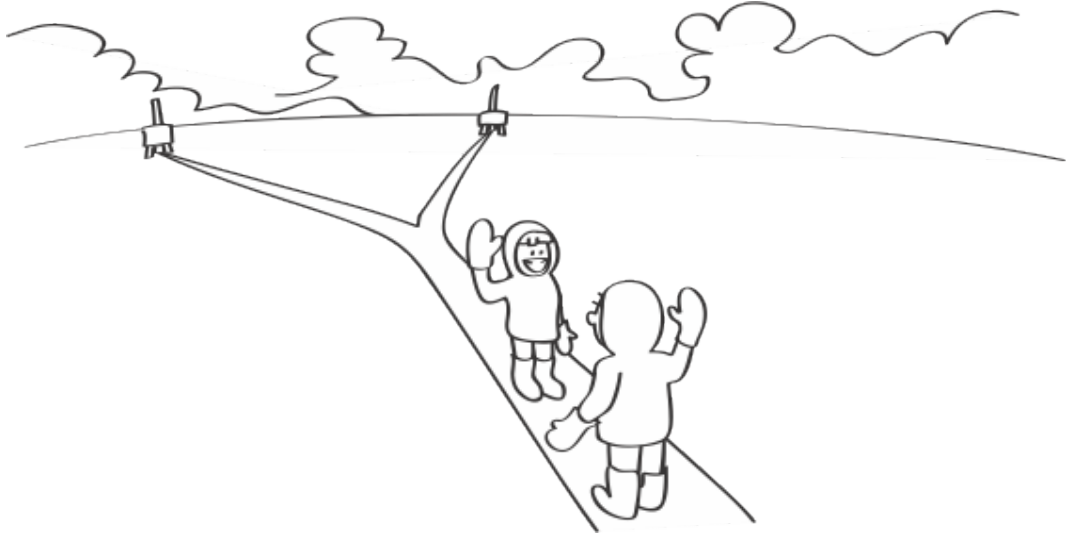
- ✓ 7 and up

Materials

Each group of students will need

- ✓ five or six pegs to place in the ground (tent pegs are good, although a coat hanger cut into pieces which are then bent over is fine),
- ✓ several meters of string or elastic,
- ✓ a ruler or tape measure, and
- ✓ pen and paper to make notes on.

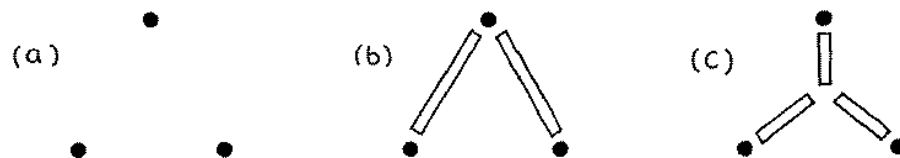
Ice Roads



Introduction

The previous activity, Tourist Town, took place in a very hot country; this one is just the opposite. In the frozen north of Canada (so the story goes), in the winter on the huge frozen lakes, snowplows make roads to connect up drill sites so that crews can visit each other. Out there in the cold they want to do a minimum of road building, and your job is to figure out where to make the roads. There are no constraints: highways can go anywhere on the snow—the lakes are frozen and covered. It's all flat.

The roads should obviously travel in straight stretches, since introducing bends would only increase the length unnecessarily. But it's not as simple as connecting all the sites with straight lines, because adding intersections out in the frozen wastes can sometimes reduce the total road length—and it's total length that's important, not travel time from one site to another.

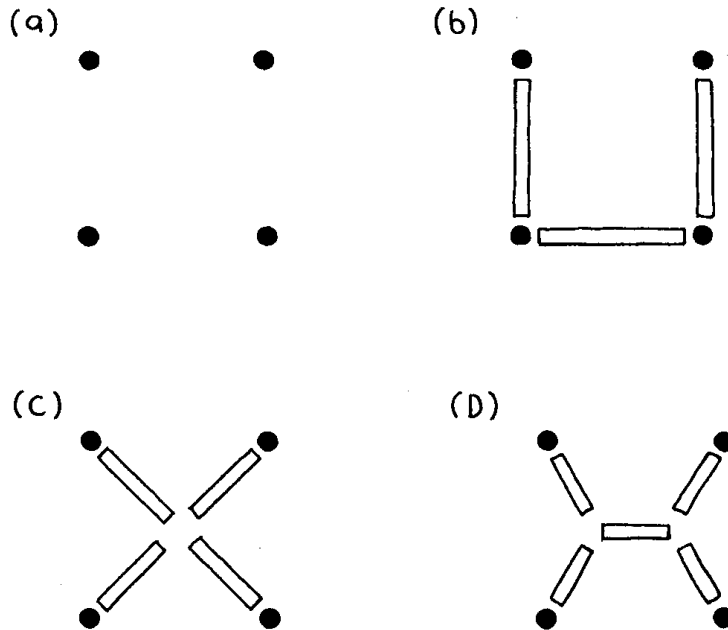


In this figure, (a) shows three drill sites. Connecting one of them to each of the others (as in (b)) would make an acceptable road network. Another possibility is to make an intersection somewhere near the center of the triangle and connect it to the three sites (c). And if you measure the total amount of road that has been cleared, this is indeed a better solution. The extra intersection is called a “Steiner” point after the Swiss mathematician

Jacob Steiner (1796–1863), who stated the problem and was the first to notice that the total length can be reduced by introducing new points. You could think of a Steiner point as a new, fictitious, drill site.

Discussion

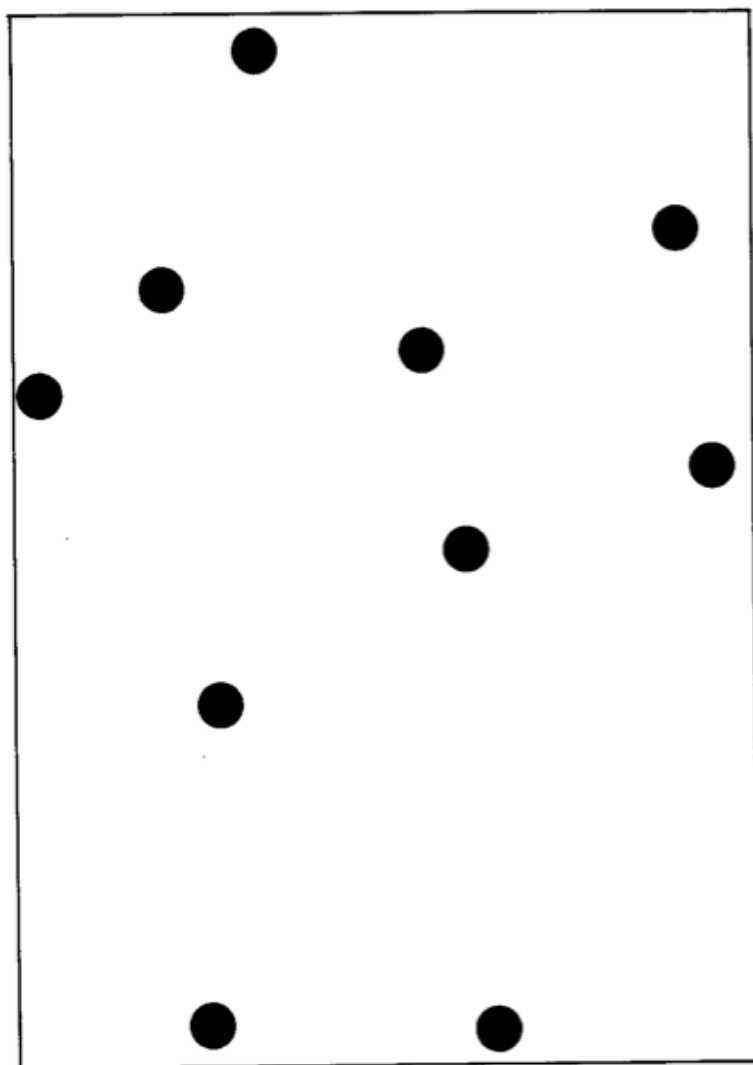
1. Describe the problem that the students will be working on. Using the example above, demonstrate to the students that with three sites, adding a new one sometimes improves the solution by reducing the amount of road-building.



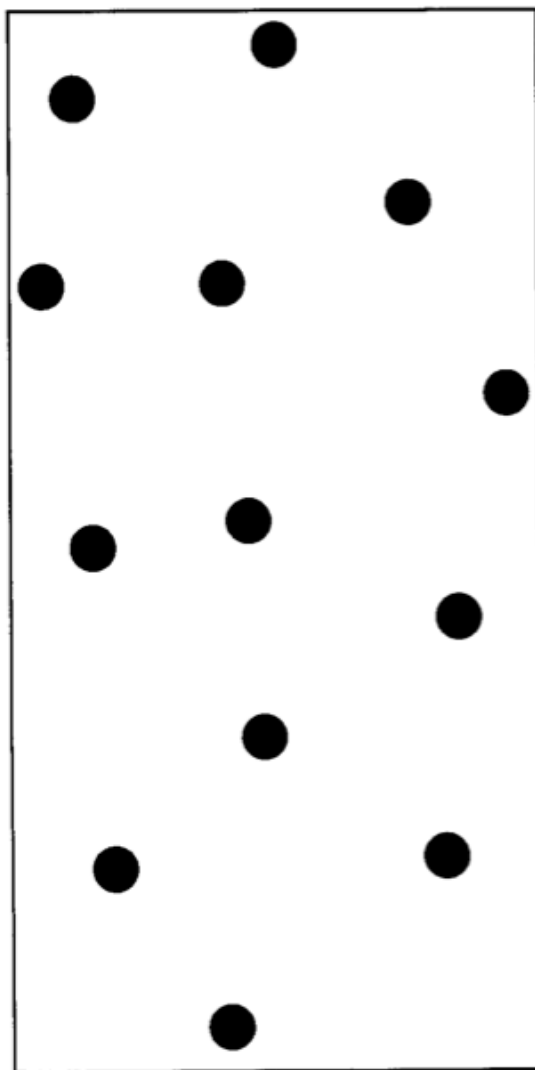
1. The students will be using four points arranged in a square, as illustrated in (a). Go outside and get each group to place four pegs in the grass in a square about 1 meter by 1 meter.
2. Get the students to experiment with connecting the pegs with string or elastic, measuring and recording the minimum total length required. At this stage they should not use any Steiner points. (The minimum is achieved by connecting three sides of the square, as in (b), and the total length is 3 meters.)
3. Now see if the students can do better by using one Steiner point. (The best place is in the center of the square, (c). Then the total length is $2\sqrt{2} = 2.83$ meters.) Suggest that they might do even better using two Steiner points. (Indeed they can, by placing the two points as in (d), forming 120 degree angles between the incoming roads. The total length is then $1 + \sqrt{3} = 2.73$ meters.)
4. Can the students do better with three Steiner points? (No – two points are best, and no advantage is gained by using more.)

5. Discuss with the students why these problems seem hard. (It's because you don't know where to put the Steiner points, and there are lots of possibilities to try out.)

Worksheet Activity: Steiner Tree Example 1



Worksheet Activity: Steiner Tree Example 2



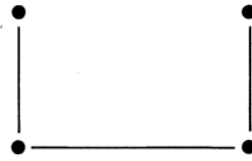
Page 132

Variations and extensions

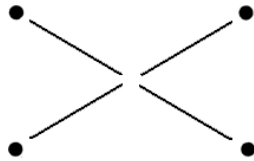
(a)



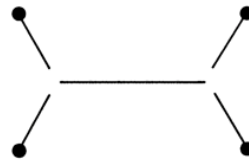
(b)



(c)

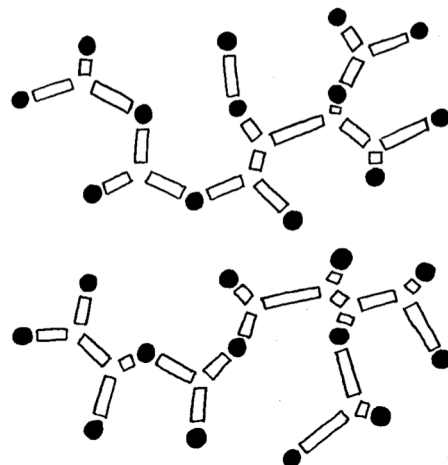


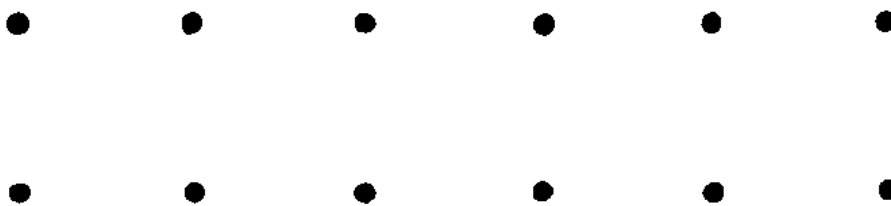
(d)



1. An interesting experiment for groups that finish the original activity early is to work with a rectangle about 1 meter by 2 meters (a). The students will find that adding one Steiner point makes things worse, but two give an improved solution. (The lengths are 4 meters for (b), $2\sqrt{5} = 4.47$ meters for (c), and $2 + \sqrt{3} = 3.73$ meters for (d).) See if they can figure out why the one-point configuration does so much worse for rectangles than for squares. (It's because when the square is stretched into a rectangle, the amount of stretch gets added just once into (b) and (d), but both diagonals increase in (c).)
2. Older students can work on a larger problem. Two layouts of sites to connect with ice roads are given in the worksheets. They can experiment with different solutions either using new copies of the worksheet, or by writing with removable pen on a transparency over the top of the sheet. Alternatively, the maps can be marked out on the ground using pegs. They can let the class know when they think they've set a new record for the shortest distance. (The figures on the right show the minimal solution for the first example and two possible solutions for the second, whose total length is quite similar.) The fact that there are two such similar solutions illustrates why these kinds of problem are so hard—there are so many choices about where to put the Steiner points!

Two possible Steiner trees for the second example

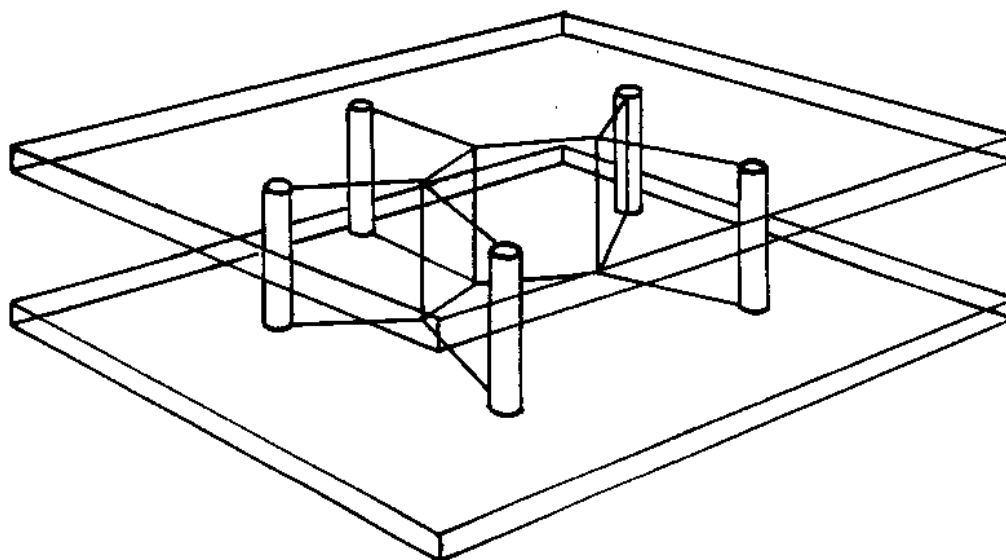




3. Ladder networks like this provide another way to extend the problem.
4. Some minimal Steiner trees for ladder networks are shown here.

The one for a two-rung ladder is just the same as for a square. However, for a three-rung ladder the solution is quite different—as you will discover if you try to draw it out again from memory! The solution for four rungs is like that for two two-rung ladders joined together, whereas for five rungs it is more like an extension of the three-rung solution. In general, the shape of the minimal Steiner tree for a ladder depends on whether it has an even or odd number of rungs. If it is even, it is as though several two-rung ladders were joined together. Otherwise, it's like a repetition of the three-rung solution. But proving these things rigorously is not easy.

Another interesting activity is to construct soap-bubble models of Steiner trees. You can do this by taking two sheets of rigid transparent plastic and inserting pins between them to represent the sites to be spanned, as shown here.



Now dip the whole thing into a soap solution. When it comes out, you will find that a film of soap connects the pins in a beautiful Steiner-tree network.

Unfortunately, however, it isn't necessarily a *minimal* Steiner tree. The soap film does find a configuration that minimizes the total length, but the

minimum is only a local one, not necessarily a global one. There may be completely different ways of placing the Steiner points to give a smaller total length. For example, you can imagine the soap film looking like the first configuration in Extension 2 when it is withdrawn from the liquid on one occasion, and the second configuration on another.

What's it all about?

The networks that we've been working on are *minimal Steiner trees*. They're called "trees" because they have no cycles, just as the branches on a real tree grow apart but do not (normally) rejoin and grow together again. They're called "Steiner" trees because new points, Steiner points, can be added to the original sites that the trees connect. And they're called "minimal" because they have the shortest length of any tree connecting those sites. In the Muddy City (Activity 9) we learned that a network connecting a number of sites that minimizes the total length is called a minimal spanning tree: Steiner trees are just the same except that new points can be introduced.

It's interesting that while there is a very efficient algorithm for finding minimal spanning trees (Activity 14)—a greedy one that works by repeatedly connecting the two closest so-far-unconnected points—there is no general efficient solution to the minimal Steiner problem. Steiner trees are much harder because you have to decide where to put the extra points. In fact, rather surprisingly, the difficult part of the Steiner tree problem is not in determining the precise location of the Steiner points, but in deciding *roughly* where to put them: the difference between the two solutions to Example 2, for example. Once you know what regions to put the new points in, fine-tuning them to the optimum position is relatively simple. Soap films do that very effectively, and so can computers.

Finding minimal Steiner trees is part of a story that involved saving big money in the telephone business. Before 1967, when corporate customers in the US operated large private telephone networks, they leased the lines from a telephone company. The amount they are billed is not calculated on the basis of how the wires are actually used, but on the basis of the shortest network that would suffice. The reasoning is that the customer shouldn't have to pay extra just because the telephone company uses a round-about route. Originally, the algorithm that calculated how much to charge worked by determining the minimal spanning tree. However, around 1967 it was noticed by a customer—an airline, in fact, with three major hubs—that if they requested a fourth hub at an intermediate point then the total length of the network would be reduced. The telephone company was forced to reduce charges to what they would have been if there was a telephone exchange at the Steiner point! Although, for typical configurations, the minimal Steiner tree is only 5% or 10% shorter than the minimal spanning tree, this can be a worthwhile saving when large amounts of money are involved. The Steiner tree problem is sometimes called the "shortest network problem" because it involves finding the shortest network that connects a set of sites.

If you have tackled the two preceding activities, the cartographer's puzzle and tourist town, you will not be surprised to hear that the minimal Steiner tree problem is NP-complete. As the number of sites increases, so does the number of possible locations for Steiner points, and trying all possibilities involves an exponentially-growing search. This is another of the thousands of problems for which it simply isn't known whether exponential search is the best that can be done, or whether there is an as-yet-undiscovered polynomial-time algorithm. What is known, however, is that if a polynomial-time algorithm is found for this problem, it can be turned into a polynomial-time algorithm for graph coloring, for finding minimal dominating sets—and for all the other problems in the NP-complete class.

We explained at the end of the previous activity that the “NP” in NP-complete stands for “non-deterministic polynomial,” and “complete” refers to the fact that if a polynomial-time algorithm is found for one of the NP-complete problems it can be turned into polynomial-time algorithms for all the others. The set of problems that are solvable in polynomial time is called P. So the crucial question is, do polynomial-time algorithms exist for NP-complete problems—in other words, is $P = NP$? The answer to this question is not known, and it is one of the great mysteries of modern computer science.

Problems for which polynomial-time algorithms exist—even though these algorithms might be quite slow—are called “tractable.” Problems for which they do not are called “intractable,” because no matter how fast your computer, or how many computers you use together, a small increase in problem size will mean that they can't possibly be solved in practice. It is not known whether the NP-complete problems—which include the cartographer's puzzle, tourist town, and ice roads—are tractable or not. But most computer scientists are pessimistic that a polynomial-time algorithm for NP-complete problems will ever be found, and so proving that a problem is NP-complete is regarded as strong evidence that the problem is inherently intractable.

What can you do when your boss asks you to devise an efficient algorithm that comes up with the optimal solution to a problem, and you can't find one?—as surely happened when the airline hit upon the fact that network costs could be reduced by introducing Steiner points. If you could *prove* that there isn't an efficient algorithm to come up with the optimal solution, that would be great. But it's very difficult to prove negative results like this in computer science, for who knows what clever programmer might come along in the future and hit upon an obscure trick that solves the problem. So, unfortunately, you're unlikely to be in a position to say categorically that no efficient algorithm is possible—that the problem is intractable. But if you can show that your problem is NP-complete, then it's actually true that thousands of people in research laboratories have worked on problems that really are

equivalent to yours, and also failed to come up with an efficient solution. That may not get you a bonus, but it'll get you off the hook!



"I can't find an efficient algorithm, I guess I'm just too dumb."

"I can't find an efficient algorithm, because no such algorithm is possible."

"I can't find an efficient algorithm, but neither can all these famous people."

What to do when you can't find an efficient algorithm: three possibilities

Of course, in real life these problems still need to be solved, and in that case people turn to *heuristics* – algorithms that don't guarantee to give the best possible solution, but do give a solution within a very small percentage of the optimal. Heuristic algorithms can be very fast, and the wastage of not finding the best possible solution can be fairly small, so they are good enough to get on with the job. It's just frustrating to know that there might be a slightly better timetable, or a slightly better layout of a network or roads.

Further reading

The cartoon is based on one in Garey and Johnson's classic book *Computers and Intractability*.

The "Computer recreations" column of *Scientific American*, June 1984, contains a brief description of how to make Steiner trees using soap bubbles, along with interesting descriptions of other analog gadgets for problem solving, including a spaghetti computer for sorting, a cat's cradle of strings for finding shortest paths in a graph, and a light-and-mirrors device for telling whether or not a number is prime. These also appear in a section about analog computers in Dewdney's *Turing Omnibus*.